

CYBER SECURITY — CASE STUDY REPORT (UNIT 4)

**AI-Powered Detection and Automated Response to
Malicious PowerShell Attacks Using Machine Learning**

Group No.: G__

Topic Selected: AI-Powered Detection and Automated Response to Malicious PowerShell
Attacks Using Machine Learning

Group Members

Sr. No.	Name of Student	Roll / Enrolment No.
1	[Student Name 1]	[Roll / Enrolment No.]
2	[Student Name 2]	[Roll / Enrolment No.]
3	[Student Name 3]	[Roll / Enrolment No.]
4	[Student Name 4]	[Roll / Enrolment No.]

Submitted to: [Name of Faculty / Course Coordinator]

[Department Name], [Institute Name]

Academic Year 2026–27

October 2026

Abstract

PowerShell is installed on every modern Windows computer, and the features that make it useful to administrators—direct access to .NET and Windows APIs, in-memory execution and remote management—also make it a favourite tool of attackers who want to “live off the land”. Signature-based antivirus copes poorly with such scripts, because a payload can be renamed, re-encoded or split into fragments without changing its behaviour. This case study designs, implements and evaluates a machine-learning pipeline that classifies PowerShell script blocks as malicious or benign and then drives a tiered automated response. The public MPSD corpus (4,202 malicious and 4,316 benign scripts) was first audited: 55% of the malicious files were exact duplicates, 12 files carried both labels, and byte-order marks appeared only in benign files. After cleaning, 1,875 malicious and 4,302 benign scripts remained, and the malicious scripts collapsed into 816 structural families. Character 3–5-gram TF-IDF features were combined with 33 interpretable statistical and behavioural indicators mapped to MITRE ATT&CK, and logistic regression, random forest, gradient boosting and a soft-voting ensemble were compared under family-aware five-fold cross-validation. The ensemble achieved an F1-score of 0.972 and a ROC-AUC of 0.9982, detecting 97.4% of malicious scripts at a 1% false-positive rate. A naive random split on the uncleaned data reported 98.8% for the same measure, showing how duplicate leakage inflates published results. Malicious code hidden inside long benign scripts proved to be the main weakness: only 46.7% of such samples were detected, rising to 96.4% when comparable examples were added to training, at the cost of a small increase in false positives. A response engine that consumes Windows Event ID 4104 records replayed an unseen fold of 1,233 scripts: it contained 91.2% of malicious scripts, wrongly contained 0.23% of benign ones and needed a median of 90.7 ms per event. The results show that lightweight, explainable models paired with a proportionate, human-supervised response policy can provide practical protection against PowerShell-based attacks.

Keywords: PowerShell; malware detection; machine learning; living-off-the-land attacks; TF-IDF; random forest; ensemble learning; MITRE ATT&CK; automated incident response; SOAR; Script Block Logging; AMSI.

1. Introduction

1.1 Background

PowerShell is the command shell and scripting language that Microsoft builds on top of the .NET runtime and installs by default on every supported version of Windows. System administrators rely on it for configuration management, Active Directory maintenance, patch deployment and cloud automation. The properties that make it valuable to them—direct access to .NET classes and Win32 APIs, the ability to run code held only in memory, and built-in remote management—are exactly what an intruder needs after gaining a foothold on a machine. A single PowerShell line can fetch a second-stage payload over HTTP, load it through reflection and run it without ever writing an executable file to disk. Security vendors call this style of intrusion “living off the land” or “fileless” because the attacker uses tools that are already trusted on the host [1].

The problem is not new, but it has not gone away. Symantec reported in 2016 that 95.4% of the PowerShell scripts submitted to the Blue Coat sandbox it analysed were malicious [2]. MITRE ATT&CK catalogues the behaviour as sub-technique T1059.001 [3], and the 2025 Threat Detection Report published by Red Canary again listed Command and Scripting Interpreter as the most frequently observed ATT&CK technique across its customer base, for the sixth year in a row [4]. Ransomware affiliates, commodity loaders and state-sponsored groups all use it.

Simply removing PowerShell is not a workable defence. In June 2022 the United States National Security Agency (NSA) and Cybersecurity and Infrastructure Security Agency (CISA), together with the national cyber security centres of New Zealand and the United Kingdom, published joint guidance that advises organisations to *keep* PowerShell and instead configure and monitor it properly [5]. Their recommendations—Script Block Logging, the Antimalware Scan Interface (AMSI), constrained language mode and transcription—produce large volumes of high-quality telemetry. What they do not provide is a decision: someone, or something, still has to look at each captured script and judge whether it is hostile, quickly enough to act before the attacker reaches their objective.

1.2 Relevance in Today’s Threat Landscape

Three trends make an AI-assisted detection and response capability for PowerShell especially timely:

- **Obfuscation is cheap and automated.** Freely available tools such as Invoke-Obfuscation can rewrite one malicious command into thousands of syntactically different but functionally equivalent forms [6]. Static signatures that match exact strings break almost immediately, while statistical models can generalise across such variants [7].
- **Security teams are overloaded.** Endpoint detection products raise far more alerts than analysts can triage, and alert fatigue is a recognised cause of missed intrusions [8]. A model that assigns a calibrated risk score lets low-risk events be handled automatically and puts the analyst’s attention where it is needed.
- **Generative AI is entering both sides of the contest.** Large language models (LLMs) have been evaluated for de-obfuscating real malware droppers [9] and for explaining why a PowerShell script is malicious [10]. Classical ML detectors remain the fast, cheap first line that decides which events deserve such deeper (and costlier) analysis.

Finally, current incident-response guidance treats detection and response as part of continuous risk management rather than a separate activity [11]. A system that links a detector to a documented, auditable response playbook therefore matches how organisations are now expected to operate.

1.3 Problem Statement

Given the text of a PowerShell script block captured on a Windows endpoint, the system must (i) estimate the probability that the script is malicious, with a false-positive rate low enough for production use, and (ii) choose a proportionate response automatically—ranging from simply logging the event to terminating the process and isolating the host—while keeping a human analyst in control of the most disruptive actions. The solution must work on static text alone, because executing unknown scripts to observe them is slow and risky, and it must be evaluated in a way that reflects how it would perform on scripts it has never seen.

1.4 Objectives

1. Audit a public, labelled PowerShell corpus for duplicates, label conflicts and spurious artefacts before any model is trained.
2. Engineer features that capture both lexical patterns and behavioural intent, and map the behavioural features to MITRE ATT&CK techniques.
3. Train and compare several classifiers using an evaluation protocol that prevents near-duplicate scripts from leaking between training and test data.
4. Design a policy-driven response engine that converts a risk score into graded actions and records every decision in an audit log.

5. Measure detection quality, response outcomes and processing latency, and discuss the limitations of the approach.

1.5 Contributions

The work makes the following contributions:

- A data-quality audit of the MPSD corpus [12] showing that 55% of its malicious files are exact duplicates, that 12 files appear under both labels, and that UTF-8 byte-order marks occur only in benign-derived files—an artefact that a model could exploit as a shortcut.
- A family-aware evaluation protocol based on structural “skeleton” hashing, which groups the 1,875 unique malicious scripts into 816 families and keeps each family entirely within one cross-validation fold.
- A hybrid detector that combines character n-gram TF-IDF features with 33 interpretable statistical and ATT&CK-aligned behavioural indicators, compared against single-view baselines.
- A tiered automated response engine that consumes Windows Event ID 4104 records, attaches ATT&CK technique context, plans containment actions and requires analyst approval before host isolation, together with an end-to-end replay experiment that measures its outcomes and latency.

1.6 Organisation of the Report

Section 2 reviews published work on PowerShell attack detection, de-obfuscation, evaluation pitfalls and automated response. Section 3 explains the methodology: data audit, pre-processing, features, models, evaluation protocol and response policy. Section 4 describes the implementation. Section 5 presents and discusses the results, and Section 6 concludes with directions for future work.

2. Literature Review

2.1 PowerShell Telemetry and the Threat Context

Since PowerShell 5.0, Microsoft has shipped several features aimed at defenders. Script Block Logging records the text of every script block that the PowerShell engine compiles in the Microsoft-Windows-PowerShell/Operational event log under Event ID 4104 [13]. Because logging happens at compile time, code that is generated at run time—for example, a string decoded from Base64 and then passed to `Invoke-Expression`—is also logged in its decoded form. AMSI goes one step further: it lets any registered anti-malware engine inspect a script buffer immediately before it runs and block it [14]. The joint NSA/CISA guidance recommends enabling both, along with transcription and constrained language mode [5]. On the analysis side, the ATT&CK knowledge base gives defenders a common vocabulary for describing adversary behaviour in terms of tactics and techniques [15], which this project uses to label what a detected script is trying to do.

Industry reporting has documented PowerShell abuse for almost a decade. Symantec’s 2016 white paper described how attackers use it for downloading payloads, lateral movement and reconnaissance [2], and its 2017 special report placed it at the centre of fileless attack techniques [1]. More recent threat-detection reports continue to rank script interpreters at the top of observed techniques [4].

2.2 Obfuscation and De-obfuscation

Bohannon’s Invoke-Obfuscation framework showed how easily PowerShell can be disguised at the token, string, encoding and launcher levels [6]. In response, Bohannon and Holmes built Revoke-Obfuscation, which extracts around 4,000 features from the abstract syntax tree (AST) of a script and uses a linear model to decide whether it is obfuscated; to build it they assembled a corpus of 408,000 scripts, about 7,000 of which were manually reviewed and labelled [7]. Their work demonstrated that *statistical* properties of code are far more robust than string signatures.

A second line of work tries to undo obfuscation before detection. PowerDrive is a multi-stage static and dynamic de-obfuscator that progressively peels off obfuscation layers and was used to analyse thousands of real attacks [16]. Li et al. proposed a light-weight, AST-subtree-based de-obfuscation method combined with semantic-aware detection [17], later extended into a more general framework by Xiong et al. [18]. PowerDP first predicts which obfuscation types were applied to a command, then uses targeted regular-expression replacement to recover the original content, and finally profiles the command’s behaviour with multi-label classifiers [19]. PowerPeeler moves to the dynamic side, de-obfuscating at the instruction level while the script executes [20]. Most recently, Patsakis et al. tested LLMs on PowerShell droppers from the Emotet campaign and found that GPT-4 could extract the hidden download URLs in 69.56% of cases [9]. The common lesson is that de-obfuscation improves detection but adds cost and complexity; in-engine telemetry such as Event ID 4104 already provides a partially de-obfuscated view at no extra cost, which is the input this project uses.

2.3 Machine-Learning Detectors for PowerShell

Hendler, Kels and Rubin carried out one of the first large studies, training character-level convolutional and recurrent networks and a natural-language-processing (NLP) baseline on 66,388 commands collected by Microsoft [21]. Their best detector was an ensemble of the NLP model and a CNN, because each caught commands the other missed. In follow-up work they used data collected through AMSI and pre-trained contextual embeddings on a large corpus of unlabelled scripts; the best model detected nearly 90% of malicious code at a false-positive rate below 0.1% [22]. Microsoft described the deployment of such deep models in its Defender products in a 2019 blog post [23].

Other studies explore different representations. Rusak et al. learned embeddings of AST nodes to classify malicious scripts by family [24]. Song et al. compared token-based and AST-based feature sets with both machine-learning and deep-learning models and reported detection rates of about 98% after feature optimisation [25]. Mimura and Tajiri showed that word-embedding representations of scripts support purely static detection [26]. Fang et al. combined FastText embeddings with textual, token and AST-node features and a random forest; they reported 98.93% accuracy on original scripts and 97.76% on a harder set in which malicious code is inserted into benign scripts, and released the dataset (MPSD) used in this case study [12]. Beyond PowerShell, Saxe and Berlin’s eXpose demonstrated that character-level models work well on short security strings such as URLs, file paths and registry keys [27]. Broader surveys of ML for malware analysis are given by Ucci et al. [28] and Gibert et al. [29]. The newest direction uses LLMs not only to classify scripts but also to point to the responsible lines of code and explain the verdict in natural language [10].

2.4 Pitfalls in Evaluating Security Machine Learning

High headline accuracy does not guarantee real-world value. Sommer and Paxson argued early on that intrusion detection differs from typical ML tasks because errors are costly, data are scarce and the environment keeps changing [30]. Arp et al. catalogued ten common pitfalls—

including sampling bias, data snooping, spurious correlations and inappropriate performance measures—and confirmed in a study of 30 papers from top security venues that these pitfalls are widespread [31]. Pendlebury et al. showed that random train/test splits which ignore time and class ratios can make malware classifiers look much better than they are [32]. For imbalanced problems, Saito and Rehmsmeier recommend complementing ROC curves with precision-oriented measures [33]. These findings directly shaped the evaluation design in Section 3.7: duplicated and near-duplicated scripts are a form of data snooping, and the byte-order-mark artefact is a textbook spurious correlation.

2.5 Automated Incident Response

Security orchestration, automation and response (SOAR) platforms connect detection tools to response actions through playbooks. Islam et al. reviewed the academic and grey literature on security orchestration and identified automation of repetitive response tasks and integration of heterogeneous tools as its main goals [34]. NoDoze reduced false alarms by ranking alerts with the help of provenance graphs [8], and Hassan et al. later showed that linking endpoint alerts into provenance graphs tagged with ATT&CK tactics and techniques reduces false alarms and helps analysts triage [35]. NIST SP 800-61 Rev. 3 organises incident-response recommendations around the six Functions of the NIST Cybersecurity Framework 2.0 (Govern, Identify, Protect, Detect, Respond and Recover), treating response as part of overall cybersecurity risk management [11].

2.6 Summary and Research Gap

Table 1 summarises representative studies. Taken together, the literature shows that (a) ML detectors can reach high accuracy on PowerShell, (b) obfuscation-robust features matter more than exact strings, and (c) evaluation choices can strongly inflate reported numbers. Three gaps remain. First, few studies audit their datasets for duplicates and artefacts before reporting results. Second, most work ends at a classification score, leaving open how that score should drive a safe, automated response. Third, response decisions are rarely tied to interpretable evidence such as ATT&CK techniques that an analyst can verify. This case study addresses all three.

Table 1: Summary of representative related work

Work	Approach	Reported outcome	Gap relevant to this study
Hendler et al. 2018 [21]	Char-level CNN/RNN + NLP ensemble	High recall at very low FPR on 66k commands	Proprietary data; detection only
Hendler et al. 2020 [22]	AMSI data, contextual embeddings	$\approx 90\%$ TPR at $< 0.1\%$ FPR	Needs large unlabelled corpus and GPU training
Bohannon & Holmes 2017 [7]	AST features + linear model	Robust obfuscation detection	Detects obfuscation, not maliciousness
Li et al. 2019 [17]	AST-subtree de-obfuscation + semantic detection	Recovers obfuscated scripts	Extra processing stage; no response
Song et al. 2021 [25]	Token/AST feature optimisation	$\approx 98\%$ detection	Random splits; no duplicate analysis
Fang et al. 2021 [12]	FastText + hybrid features + RF	98.93% / 97.76% accuracy	Duplicates in data not addressed
Tsai et al. 2023 [19]	Obfuscation-type prediction + regex de-obfuscation	98.11% recovery over 15 types	Command-level; no response
Wang et al. 2026 [10]	LLM classification + explanation	Human-readable explanations	Cost and latency of LLM inference

3. Methodology

3.1 Overview

Figure 1 shows the overall design. Script-block text arrives from endpoint telemetry (Event ID 4104 or an AMSI buffer). It is cleaned and normalised, converted into two complementary feature views, and scored by a trained model. A policy engine converts the risk score and the behavioural evidence into a response tier, and the corresponding actions are executed (or, in this study, simulated) and logged. Analyst decisions feed back into the labelled data used for periodic retraining. The offline part of the pipeline—data audit, model training, cross-validation and threshold selection—is described in Sections 3.3 to 3.7; the online response logic is described in Section 3.8.

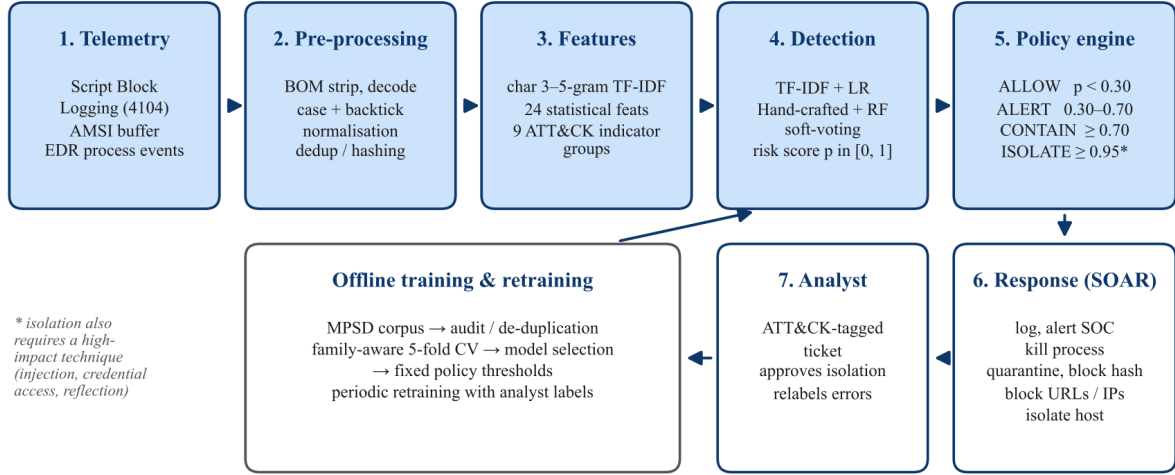


Figure 1: End-to-end architecture of the proposed detection and automated response system

3.2 Threat Model and Assumptions

The adversary has already obtained code execution on a Windows endpoint (for example, through a phishing attachment or a stolen credential) and uses PowerShell to download tools, inject code, establish persistence, dump credentials or open a command channel. The adversary may obfuscate scripts, rename variables and embed malicious code inside otherwise legitimate scripts. We assume Script Block Logging or AMSI is enabled and that its output reaches the analysis service; an attempt to disable logging is itself treated as a suspicious signal (the defence-evasion indicator group in Section 3.5.3). Out of scope are attacks that never invoke PowerShell and white-box adversarial attacks in which the attacker can query or inspect the model to craft evasive inputs [36]; the latter are discussed as a limitation.

3.3 Dataset and Data-Quality Audit

The study uses the public MPSD corpus released by Fang et al. [12]. It contains 4,202 malicious scripts (malicious_pure), 4,316 benign scripts (powershell_benign_dataset) and 4,202 “mixed” malicious scripts, each created by inserting one malicious script into one benign script. The malicious samples cover download cradles, shellcode loaders, reverse shells, credential-theft tools and persistence scripts; the benign samples are administration scripts and modules collected from public repositories.

Before training, the corpus was audited using three checks recommended by the security-ML literature [31]:

1. **Exact duplicates.** Each script was hashed (SHA-256) after collapsing whitespace and lowercasing. Only 1,875 of the 4,202 malicious files were unique: 55.2% of them were exact copies of another file. Duplicates were removed so that each distinct script is counted once.
2. **Label conflicts.** 12 files, sharing 6 distinct script contents, appeared in both the malicious and the benign folders. Since their true label cannot be known, all of them were discarded.
3. **Spurious artefacts.** None of the malicious files started with a UTF-8 byte-order mark (BOM), whereas about 35% of the benign files did. A model could learn “BOM means benign”—a rule that an attacker could exploit trivially by adding three bytes. The BOM is therefore stripped from every file before any feature is computed.

Removing exact duplicates is not sufficient, because many malicious scripts are produced by the same generator and differ only in variable names, numeric constants (such as port numbers

or shellcode bytes) or whitespace. To capture these near-duplicates, each script is reduced to a structural *skeleton*: it is lowercased, every variable name is replaced by a placeholder, every decimal or hexadecimal literal is replaced by 0, and all whitespace is removed. Scripts with the same skeleton hash form one *family*. The 1,875 unique malicious scripts collapse into only 816 families, while the 4,302 benign scripts form 4,292 families. Figure 2 visualises the effect of each cleaning step.

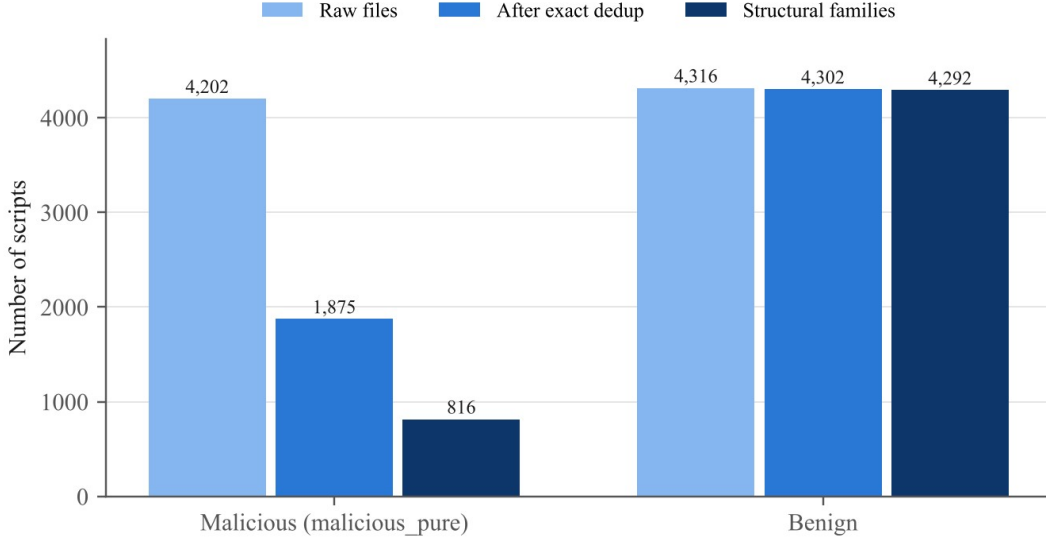


Figure 2: Effect of the data-quality audit: raw files, unique scripts after exact de-duplication, and structural families

3.4 Pre-processing and Normalisation

Each script is decoded as UTF-8 (or UTF-16 if a UTF-16 BOM is present), the BOM is removed and the text is truncated to 200,000 characters to bound processing time for a small number of very large files. For the lexical view, the text is then normalised in two ways that do not change the meaning of PowerShell code: it is converted to lower case, because PowerShell keywords, cmdlets and variable names are case-insensitive, and backtick characters are removed, because PowerShell ignores the backtick escape inside identifiers (so that `I`nv`oke-Ex`pression` and `Invoke-Expression` are the same command). Runs of spaces and tabs are collapsed. These steps cheaply undo two of the most common token-level obfuscations without any execution. The statistical features in Section 3.5.2 are computed on the raw text, because properties such as case ratio and backtick density are themselves informative.

3.5 Feature Engineering

Two complementary views of each script are built. The lexical view captures *how the code is written*; the statistical and behavioural view captures *what the code does* and *what it looks like overall*. Using both follows the observation by Hendler et al. that models built on different representations make different mistakes [21].

3.5.1 Character *n*-gram TF-IDF

The normalised text is split into overlapping character sequences of length 3, 4 and 5. Character *n*-grams are robust to renamed variables and partly split strings, and they do not require a PowerShell parser. Each *n*-gram *t* in script *d* is weighted with sub-linear term frequency and smoothed inverse document frequency [37]:

$$w_{t,d} = (1 + \ln tf_{t,d}) \times [\ln((1 + N) / (1 + df_t)) + 1] \quad (1)$$

where $tf_{t,d}$ is the number of times t occurs in d , N is the number of training scripts and df_t is the number of training scripts containing t . Each script vector is then scaled to unit Euclidean length. Only n -grams that occur in at least three training scripts are kept, and the vocabulary is capped at the 50,000 most frequent n -grams. The vectoriser is fitted inside each cross-validation fold on training data only, so that document frequencies never include test scripts.

3.5.2 Statistical Features

Twenty-four numeric features describe the overall shape of a script: its length and number of lines (log-scaled), mean and maximum line length, the proportions of alphabetic, digit, upper-case, whitespace and special characters, the densities of backticks, plus signs (used for string concatenation), quotes and braces, the length of the longest token, the number of Base64-like blobs (runs of 80 or more Base64 characters), the number of URLs and IPv4 addresses, the density and number of distinct variables, the proportion of comment lines, the number of function definitions, the number of comment-based help keywords (`.SYNOPSIS`, `.EXAMPLE` and so on) and whether a `param()` block is present. Character-level Shannon entropy [38] is included because encoded or compressed payloads have higher entropy than hand-written code:

$$H(d) = - \sum_c p_c \log_2 p_c \quad (2)$$

where p_c is the relative frequency of character c in the script. The documentation-related features reflect a simple observation: legitimate administrative scripts are usually written to be maintained by others and therefore contain help blocks, parameters and functions, whereas attack scripts are usually short, dense and undocumented.

3.5.3 Behavioural Indicator Groups Mapped to ATT&CK

Nine further features count matches for groups of regular expressions that capture common attacker behaviours (Table 2). Each count is log-scaled. Grouping patterns by tactic rather than using each keyword as a separate feature keeps the feature space small and stable, and—more importantly—gives the response engine human-readable evidence: when a script is blocked, the alert can state, for example, that it combined a download cradle (T1105) with dynamic execution (T1059.001). In total the statistical and behavioural view has 33 features.

Table 2: Behavioural indicator groups, example patterns and the ATT&CK technique each group supports

Indicator group	Example patterns (normalised text)	ATT&CK technique
Dynamic execution	invoke-expression, iex, invoke-command, [scriptblock]::create	T1059.001 PowerShell
Download cradle	downloadstring, downloadfile, net.webclient, invoke-webrequest, bitstransfer	T1105 Ingress Tool Transfer
Encoding / compression	frombase64string, -encodedcommand, -bxor, gzipstream, [char]NN	T1027 Obfuscated Files or Information
Win32 API / injection	virtualalloc, createthread, writeprocessmemory, dllimport, kernel32	T1055 Process Injection
Defence evasion	-windowstyle hidden, -nopprofile, bypass, amsi, set-mppreference	T1562.001 Disable or Modify Tools
Persistence	schtasks, register-scheduledtask, currentversion\run, new-service	T1053.005 Scheduled Task
Credential access	mimikatz, sekurlsa, lsass, logonpasswords	T1003.001 LSASS Memory
Raw sockets	net.sockets, tcpclient, tcplistener, .getstream()	T1095 Non-Application Layer Protocol
Reflection	reflection.assembly, ::load(, add-type	T1620 Reflective Code Loading

3.6 Classification Models and Justification

Four single-view models and one ensemble were compared. All are implemented with scikit-learn [39] and use class weighting to compensate for the roughly 1:2.3 malicious-to-benign ratio after cleaning.

- **Char TF-IDF + Logistic Regression (LR).** A linear model is the natural choice for very high-dimensional sparse text features: it trains in seconds, is hard to over-fit with L2 regularisation (inverse strength $C = 10$ here) and its coefficients can be read directly as the n-grams that push a script towards one class. The model estimates $P(\text{malicious} | x) = \sigma(w \cdot x + b)$, where σ is the logistic function.
- **Hand-crafted + LR.** The same linear model on the 33 standardised statistical and behavioural features. It serves as an interpretable baseline that shows how much can be achieved with a few dozen features.
- **Hand-crafted + Random Forest (RF).** An ensemble of 300 decision trees trained on bootstrap samples with random feature subsets [40]. RFs capture non-linear interactions (for example, “high entropy *and* no comments”), need no feature scaling and are robust to outliers such as extremely long scripts.
- **Hand-crafted + Histogram Gradient Boosting (HGB).** A boosted-tree model that bins features into histograms for speed, in the style of LightGBM [41]. It is included as the strongest conventional tabular learner.
- **Soft-voting ensemble.** The final detector averages the malicious-class probabilities of the lexical model and the behavioural RF:

$$p_{\text{ens}}(x) = \frac{1}{2} \cdot p_{\text{LR}}^{\text{TF-IDF}}(x) + \frac{1}{2} \cdot p_{\text{RF}}^{\text{hand}}(x) \quad (3)$$

The two members look at different evidence—surface character patterns versus overall shape and behaviour—so their errors are only weakly correlated, and averaging them reduces variance. Equal weights were chosen deliberately instead of tuning them on the test folds, to avoid optimistic bias. Deep neural networks were not used: on a corpus of about six thousand unique scripts, published results [12], [25] suggest that well-engineered classical models are competitive, and they are cheaper to train, faster at inference and easier to explain to an analyst.

3.7 Evaluation Protocol

All models are evaluated with five-fold cross-validation [42] under two protocols:

- **Naive protocol (for comparison only).** The raw corpus, including duplicates, is split with ordinary stratified five-fold cross-validation. This mimics a common practice in the literature and shows how optimistic it can be.
- **Family-aware protocol (main results).** The de-duplicated corpus is split with stratified *group* five-fold cross-validation, using the skeleton family from Section 3.3 as the group. Every member of a family falls into the same fold, so a model is never tested on a script whose near-twin it saw during training.

Predictions from the held-out folds are pooled (out-of-fold predictions) so that every script is scored exactly once by a model that did not see it. With TP, FP, TN and FN denoting true/false positives/negatives and malicious as the positive class, the following measures are reported:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}), \quad \text{Recall (TPR)} = \text{TP} / (\text{TP} + \text{FN}) \quad (4)$$

$$\text{F1} = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall}), \quad \text{FPR} = \text{FP} / (\text{FP} + \text{TN}) \quad (5)$$

together with accuracy, the area under the ROC curve (ROC-AUC) and the true-positive rate at a fixed false-positive rate of 1% (TPR@1%FPR). The last measure matters most operationally: in a large organisation, even a 1% false-positive rate on benign scripts can generate many alerts per day, so the detection rate *at a low, fixed FPR* is a more honest summary than accuracy [31], [33].

A third experiment stresses the detector with the mixed subset of MPSD, in which malicious code is hidden inside a benign script. To avoid leakage, a mixed sample is scored only by the model of the fold that did *not* contain its malicious parent script; the benign host script may have been seen during training as benign, which makes the test deliberately harder. Mixed samples whose malicious parent could not be identified from the file name were excluded.

3.8 Automated Response Design

Detection alone does not stop an attack. The response engine turns the ensemble’s score p and the set G of behavioural indicator groups that fired into one of four tiers. The tiers follow the principle of proportionality: cheap, reversible actions are automated freely; disruptive actions require stronger evidence; and the most disruptive action—isolating a host from the network—additionally requires analyst approval.

Table 3: Response policy: tiers, triggering conditions and automated actions

Tier	Condition	Automated actions	Human role
ALLOW	$p < 0.30$	Record verdict in SIEM	None
ALERT	$0.30 \leq p < 0.70$	Record verdict; open SOC ticket with score, ATT&CK techniques and defanged IOCs	Analyst triages
CONTAIN	$p \geq 0.70$	All of the above, plus terminate the PowerShell process, quarantine the script file, block its SHA-256 hash, block extracted URLs/IPs	Analyst reviews and may roll back
ISOLATE	$p \geq 0.95$ and G contains injection, credential access or reflection	All of the above, plus network isolation of the host	Approval required before isolation

Table 3 lists the policy. The thresholds were set *a priori* from the operational meaning of each tier rather than tuned on test data; Section 5 reports how the out-of-fold scores fall relative to them. Every decision—score, tier, techniques, indicators of compromise (IOCs) and actions—is written as one JSON line to an append-only audit log. IOCs are “defanged” (for example, http becomes hxxp and dots become [.]) so that logs cannot be clicked accidentally. By default the engine runs in *dry-run* mode, marking every action as “simulated”; this mirrors how SOAR playbooks are normally validated in production before they are allowed to act [34].

3.9 Ethical and Safety Considerations

All malicious samples were handled purely as text and were never executed. The dataset is downloaded from its public source at run time and is not redistributed in the project repository. IOCs are defanged in all outputs and are omitted from this report. The response engine cannot perform any real action unless dry-run mode is explicitly switched off, and host isolation always requires human approval. These choices follow the principle that an automated defence should fail safe and remain accountable to people [11].

4. Implementation Details

4.1 Development Environment

Table 4: Software and hardware environment

Component	Version / specification
Language	Python 3.13
Machine-learning library	scikit-learn 1.9 [39]
Data handling	pandas 3.0, NumPy 2.5
Plotting	Matplotlib 3.11
Testing	pytest (7 unit tests)
Hardware	4 virtual CPU cores, 15 GB RAM, no GPU (Linux container)
Dataset	MPSD, public GitHub repository das-lab/mpsd [12]

4.2 Software Architecture

The code is organised as a small Python package plus experiment scripts (Table 5). Every model is a scikit-learn Pipeline that accepts raw script text, so exactly the same code path is used during cross-validation, in the response simulation and in a deployment. All random seeds are fixed (seed 42) so that every number in this report can be regenerated with three commands: fetch the data, run the experiments and build the figures.

Table 5: Main modules of the implementation

Module	Responsibility
psdetect/data.py	Reads scripts, strips BOMs, removes exact duplicates and label conflicts, computes skeleton families
psdetect/features.py	Normalisation, 24 statistical features, 9 ATT&CK-aligned indicator groups, scikit-learn transformer
psdetect/models.py	Model pipelines (TF-IDF + LR, hand-crafted + LR/RF/HGB) and the soft-voting ensemble
psdetect/response.py	Response engine: scoring, tier selection, ATT&CK mapping, IOC extraction and defanging, action planning, audit log
experiments/ run_experiments.py	Naive and family-aware cross-validation, mixed-script stress test, feature importance
experiments/ simulate_response.py	Replays a held-out fold as Event ID 4104 records through the response engine
experiments/make_figures.py	Generates all figures in this report from the saved results
tests/test_pipeline.py	Unit tests for normalisation, features, family hashing, policy tiers and defanging

4.3 Hyper-parameters

Table 6 lists the hyper-parameters. They were fixed from common defaults and a brief sanity check, not searched exhaustively on the evaluation folds; a nested search would likely add a little performance but would also add a risk of over-fitting the small number of malicious families.

Table 6: Model hyper-parameters

Model	Key settings
Char TF-IDF + LR	n-gram range 3–5 (characters), min_df = 3, max_features = 50,000, sub-linear tf, LR with C = 10, L2 penalty, class_weight = balanced, max_iter = 2,000
Hand-crafted + LR	Standard scaling; C = 1, class_weight = balanced
Hand-crafted + RF	300 trees, Gini impurity, unlimited depth, min_samples_leaf = 1, class_weight = balanced
Hand-crafted + HGB	300 boosting iterations, learning rate 0.08, class_weight = balanced
Ensemble	Equal-weight average of TF-IDF + LR and hand-crafted + RF probabilities

4.4 Response Engine Algorithm

For each incoming Event ID 4104 record, the engine performs the following steps:

1. Read ScriptBlockText, ScriptBlockId, Computer, ProcessId and Path from the event.
2. Compute the ensemble probability p for the script text.
3. Normalise the text and count matches for each indicator group; let G be the set of groups with at least one match.
4. Select the tier: ISOLATE if $p \geq 0.95$ and G contains a high-impact group; otherwise CONTAIN if $p \geq 0.70$; otherwise ALERT if $p \geq 0.30$; otherwise ALLOW.
5. Map every group in G to its ATT&CK technique; for ALERT and above, extract and defang URLs and IP addresses.
6. Build the action plan for the tier (Table 4); mark each action as simulated or executed according to the dry-run setting; flag ISOLATE for analyst approval.
7. Measure the elapsed time, write the complete decision as one JSON line to the audit log and return it.

4.5 Deployment Path on Windows Endpoints

In a real deployment, Script Block Logging is enabled through Group Policy (Administrative Templates → Windows Components → Windows PowerShell → Turn on PowerShell Script Block Logging) [13]. Event ID 4104 records are shipped to a central collector with Windows Event Forwarding or an endpoint agent. Large scripts are split by Windows across several 4104 events that share a ScriptBlockId and carry MessageNumber and MessageTotal fields, so the collector reassembles them before scoring. The trained pipeline is serialised with joblib and served behind a small internal API; the response engine’s action stubs are then connected to the organisation’s endpoint detection and response (EDR) platform, firewall and ticketing system. Because AMSI exposes the same script text immediately before execution [14], the identical model could also be called from an AMSI provider to block scripts *before* they run rather than responding after the fact.

4.6 Testing

Seven unit tests check that normalisation folds case and backticks, that the feature vector has the expected shape and contains only finite values, that indicator groups fire on a typical download cradle but not on a documented administrative function, that two scripts differing only in variable names share a skeleton hash, that the policy produces the correct tier for given scores, that isolation is chosen only when a high-impact technique is present and is flagged for approval, and that IOC defanging works. All tests pass.

5. Results and Discussion

5.1 Impact of the Data-Quality Audit

The audit changed the experimental picture before any model was trained. Of 8,518 raw files, 2,329 exact duplicates and 12 conflicting files were removed, leaving 1,875 malicious and 4,302 benign scripts. The class balance therefore shifted from roughly 1:1 in the raw corpus to about 1:2.3 after cleaning. More importantly, the 1,875 unique malicious scripts belong to only 816 structural families, so on average each malicious “idea” appears about 2.3 times with cosmetic changes. Any split that scatters such families across training and test folds lets a model succeed by recognising near-copies rather than by learning what makes a script malicious. Section 5.3 measures how large that effect is.

5.2 Detection Performance under Family-Aware Evaluation

Table 7 reports the main results. Every model performs well, which confirms that malicious and benign PowerShell differ substantially in both wording and behaviour. The differences between models are nevertheless informative.

Table 7: Out-of-fold detection performance under family-aware five-fold cross-validation (threshold 0.5; malicious = positive class)

Model	Acc.	Prec.	Recall	F1	ROC-AUC	TPR @1% FPR	FP	FN
Handcrafted + LR	0.9736	0.9534	0.9600	0.9567	0.9884	0.9413	88	75
Handcrafted + RF	0.9801	0.9818	0.9520	0.9667	0.9953	0.9568	33	90
Handcrafted + HGB	0.9794	0.9839	0.9477	0.9655	0.9957	0.9621	29	98
Char TF-IDF + LR	0.9845	0.9800	0.9685	0.9742	0.9986	0.9733	37	59
Ensemble (TF-IDF LR + RF)	0.9835	0.9836	0.9616	0.9725	0.9982	0.9739	30	72

- **Lexical features are the strongest single view.** Char TF-IDF + LR achieves the highest F1 (0.9742) and ROC-AUC (0.9986). Character n-grams pick up fragments of API names, .NET type names and command-line switches even when they are split across strings or surrounded by unfamiliar code.
- **Thirty-three interpretable features go a long way.** With only the hand-crafted view, the tree models reach F1 of 0.9667 (RF) and 0.9655 (HGB) and the highest precision of all models (0.9818 and 0.9839). The linear model on the same features is clearly weaker (F1 0.9567, 88 false positives), which shows that interactions between features—captured by trees but not by a linear boundary—matter.
- **The ensemble trades a little recall for fewer false alarms.** At the default threshold the ensemble produces 30 false positives versus 37 for the lexical model, at the cost of 13 additional false negatives, giving an almost identical F1 (0.9725 vs 0.9742). At the operating point that matters most—1% FPR—the two are virtually identical (97.39% vs 97.33%).

The ensemble was nonetheless kept as the detector behind the response engine for two practical reasons. First, false positives are what make automated containment expensive—each one can kill a legitimate administrator’s job—so the model with the fewest false alarms at a competitive detection rate is preferable. Second, its Random Forest member contributes behavioural evidence that the response engine can explain to an analyst more clearly than thousands of n-gram weights. Across the five folds, the ensemble’s F1 ranged from 0.945 to 0.994 (mean 0.972, standard deviation 0.017); the spread comes mostly from which malicious families happen to fall into each test fold, and the result is not driven by one lucky split.

Figure 3 shows the ROC curves on a logarithmic false-positive axis, which magnifies the low-FPR region that matters in practice. The lexical model and the ensemble dominate the hand-crafted models below 1% FPR, while the hand-crafted linear model falls behind across the whole range.

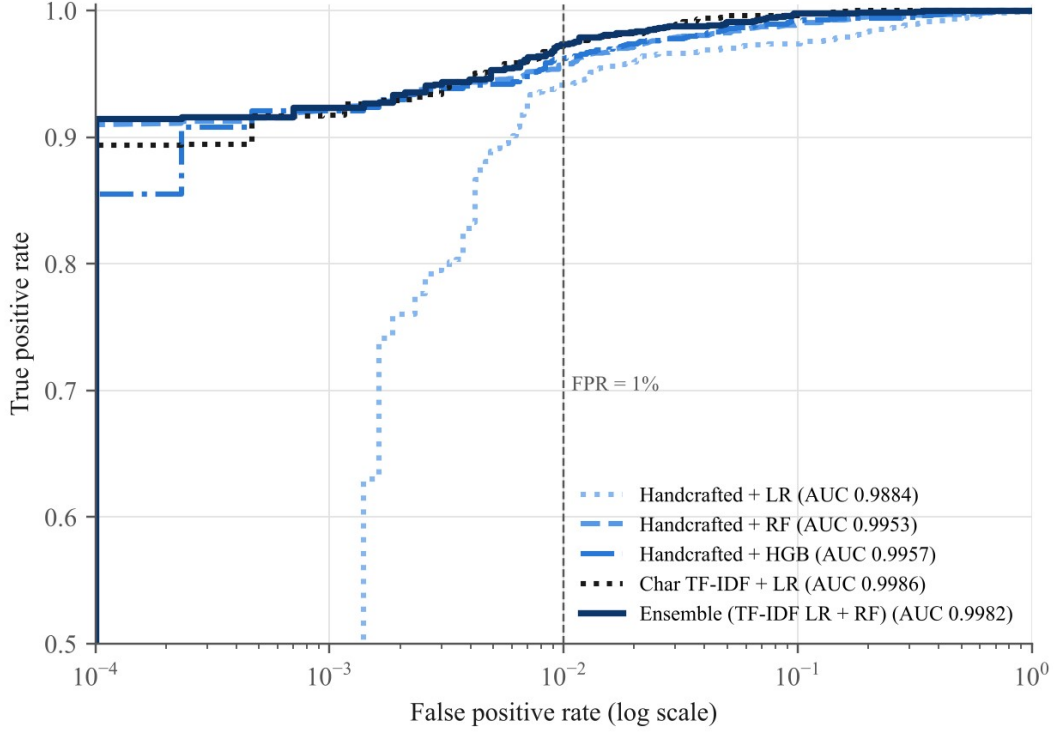


Figure 3: ROC curves of all models under family-aware cross-validation (logarithmic false-positive axis; the dashed line marks FPR = 1%)

Figure 4 gives the ensemble’s confusion matrix: 4,272 of 4,302 benign scripts (99.30%) and 1,803 of 1,875 malicious scripts (96.16%) are classified correctly.

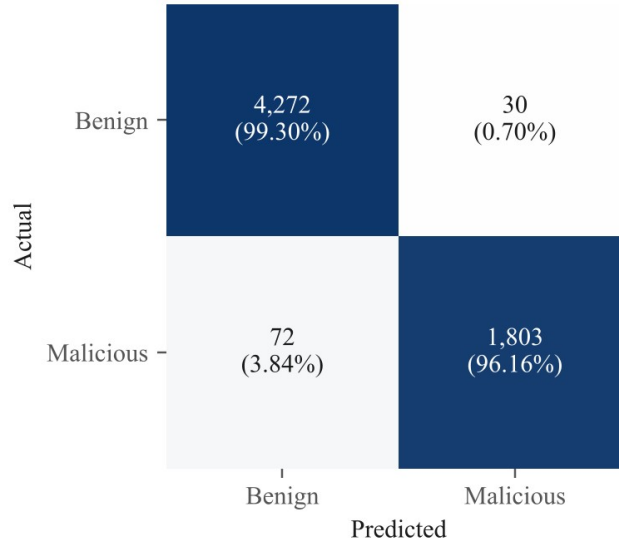


Figure 4: Confusion matrix of the ensemble (out-of-fold, threshold 0.5); percentages are per actual class

5.3 Effect of the Evaluation Protocol

Table 8 and Figure 5 compare the two protocols. Under the naive protocol—duplicates kept, random folds—every model looks better: F1 rises by 0.014 to 0.024 points, and the ensemble’s TPR at 1% FPR rises from 97.39% to 98.83%. These gaps look small in absolute terms, but they hide a large relative difference in errors: the share of malicious scripts that the ensemble misses is 2.3 times higher under the family-aware protocol than under the naive one. In other words, a

naive evaluation would understate the miss rate a security team should expect on genuinely new attacks by a factor of about 2. This mirrors the warnings of Arp et al. [31] and Pendlebury et al. [32] and suggests that results reported on MPSD with random splits, including very high accuracies, should be read with this caveat in mind.

Table 8: Naive random split on raw data versus family-aware split on cleaned data

Model	F1 (naive)	F1 (family-aware)	TPR @ 1% FPR (naive)	TPR @ 1% FPR (family-aware)
Handcrafted + LR	0.9807	0.9567	0.9700	0.9413
Handcrafted + RF	0.9853	0.9667	0.9831	0.9568
Handcrafted + HGB	0.9859	0.9655	0.9831	0.9621
Char TF-IDF + LR	0.9881	0.9742	0.9879	0.9733
Ensemble (TF-IDF LR + RF)	0.9880	0.9725	0.9883	0.9739

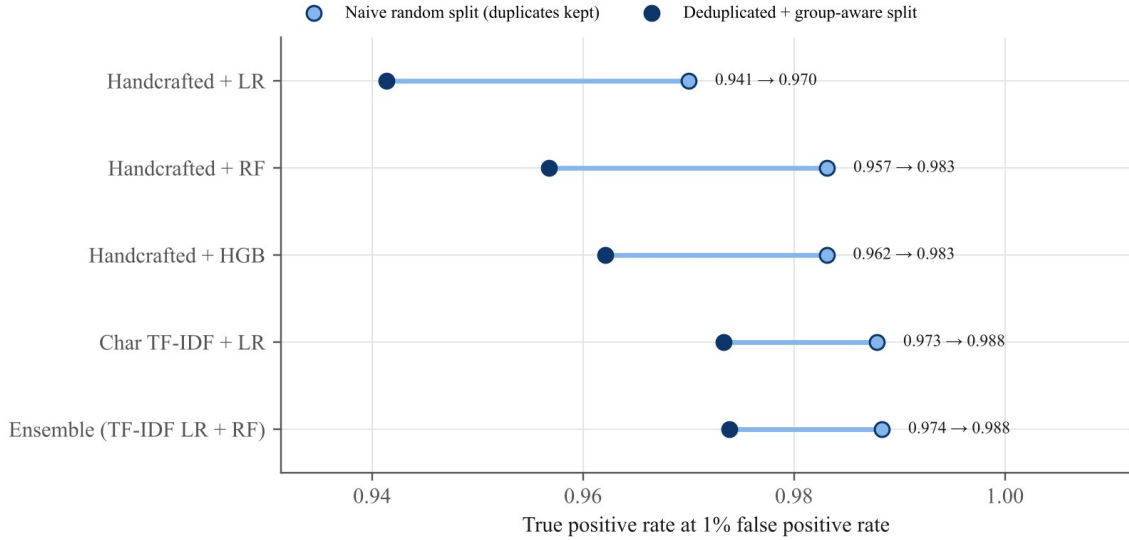


Figure 5: True-positive rate at 1% false-positive rate under the two evaluation protocols (labels show family-aware → naive)

5.4 Stress Test: Malicious Code Embedded in Benign Scripts

The mixed subset of MPSD represents a realistic evasion strategy: hiding a payload inside a long, legitimate-looking administration script. 4,064 mixed scripts were scored (138 were skipped because their malicious parent could not be identified). Table 9 shows the results. The best detection rate was obtained by Handcrafted + LR (53.42%) and the lowest by Handcrafted + HGB (41.07%); the ensemble detected 46.68%.

Table 9: Detection rate on mixed scripts (malicious code inserted into a benign script), scored by models that never saw the malicious parent

Model	Detection rate ($p \geq 0.5$)	Mean risk score
Handcrafted + LR	53.42%	0.539
Handcrafted + RF	47.12%	0.397
Handcrafted + HGB	41.07%	0.405
Char TF-IDF + LR	48.99%	0.500
Ensemble (TF-IDF LR + RF)	46.68%	0.448

This is the most important weakness uncovered by the study: every model trained only on standalone scripts misses roughly half of the embedded payloads. The explanation follows from Section 5.5. The strongest signals are the overall *shape* of a script and, for the lexical model, its length-normalised n-gram profile. When a few lines of malicious code are buried inside hundreds of lines of neatly formatted administrative code, both views are dominated by the benign host. The simplest model, logistic regression on hand-crafted features, does best because its indicator counts add up linearly instead of being overruled by shape-based splits, as happens in the tree models.

5.4.1 Mitigation: mixed-aware training

Following Fang et al., who included mixed scripts in training [12], the training folds were augmented with mixed samples. To prevent leakage, a mixed script was added to the training data of fold k only when *both* its malicious parent and its benign host belonged to the training folds (on average 2,591 scripts per fold); mixed scripts whose malicious parent lay in fold k were again held out for testing. Table 10 shows the effect. For the ensemble, the detection rate on embedded payloads moved from 46.7% to 96.4%, and 98.3% of them now reach at least the ALERT tier, while its F1 on ordinary scripts changed by -0.004 and its false-positive rate by $+0.35$ percentage points. The best mixed-script detection after augmentation was achieved by Handcrafted + HGB (97.4%). Adding realistic examples of embedded code to the training data therefore closes most of the gap. The price is a modest rise in false alarms on ordinary scripts (from 0.70% to 1.05%): having seen benign host code inside malicious-labelled samples, the model becomes slightly more suspicious of similar benign scripts. Because the tiered policy routes borderline scores to ALERT rather than to automatic containment, this trade-off is worthwhile, and mixed-aware training is recommended for the deployed model, with the false-positive rate re-measured on local data.

Table 10: Effect of adding mixed scripts to the training folds (base $\rightarrow$ mixed-aware); clean-test metrics are out-of-fold on the de-duplicated corpus

Model	Clean F1	Clean FPR	Mixed detected ($p \geq 0.5$)	Mixed $\geq$ ALERT
Handcrafted + LR	0.957 $\rightarrow$ 0.935	2.05% $\rightarrow$ 3.67%	53.4% $\rightarrow$ 94.1%	97.3%
Handcrafted + RF	0.967 $\rightarrow$ 0.960	0.77% $\rightarrow$ 1.51%	47.1% $\rightarrow$ 96.6%	98.5%
Handcrafted + HGB	0.965 $\rightarrow$ 0.960	0.67% $\rightarrow$ 1.53%	41.1% $\rightarrow$ 97.4%	97.6%
Char TF-IDF + LR	0.974 $\rightarrow$ 0.969	0.86% $\rightarrow$ 1.12%	49.0% $\rightarrow$ 92.1%	96.1%
Ensemble (TF-IDF LR + RF)	0.972 $\rightarrow$ 0.969	0.70% $\rightarrow$ 1.05%	46.7% $\rightarrow$ 96.4%	98.3%

5.5 Explainability

Figure 6 lists the most influential hand-crafted features of the Random Forest. The five most important are whitespace ratio, line count, special-character ratio, mean line length and digit ratio—all measures of the *shape* of the code rather than of particular commands. The pattern is easy to interpret: attack scripts in the corpus are usually dense one-liners or short blocks with long lines, little indentation, many special characters and long runs of digits (for example, shellcode stored as byte arrays), whereas administration scripts are spread over many short, indented lines. The first behavioural feature, the Win32 API/injection indicator group, ranks eighth. The dominance of shape features is a double-edged result: they are cheap and effective, but an attacker can change them simply by re-formatting the code or by wrapping it in a long benign script, which is exactly the weakness exposed in Section 5.4.

The logistic-regression model on character n-grams offers a complementary, more behavioural explanation. Among its highest positive weights are “iex”, “(new-”, “w-ob”, “.exe” and “dow”—fragments of the Invoke-Expression alias, of New-Object, of executable file names and of Download methods—whereas the strongest benign n-grams, such as “·{·\$”, “\n}·” and “name”, reflect neatly formatted code blocks and named parameters (here “·” marks a space and “\n” a line break). For an analyst, the most useful explanation is the list of ATT&CK techniques that the response engine attaches to every alert (Section 5.7), because it describes *behaviour* rather than model internals. Model-agnostic attribution methods such as SHAP [43] could add per-script explanations in a future version.

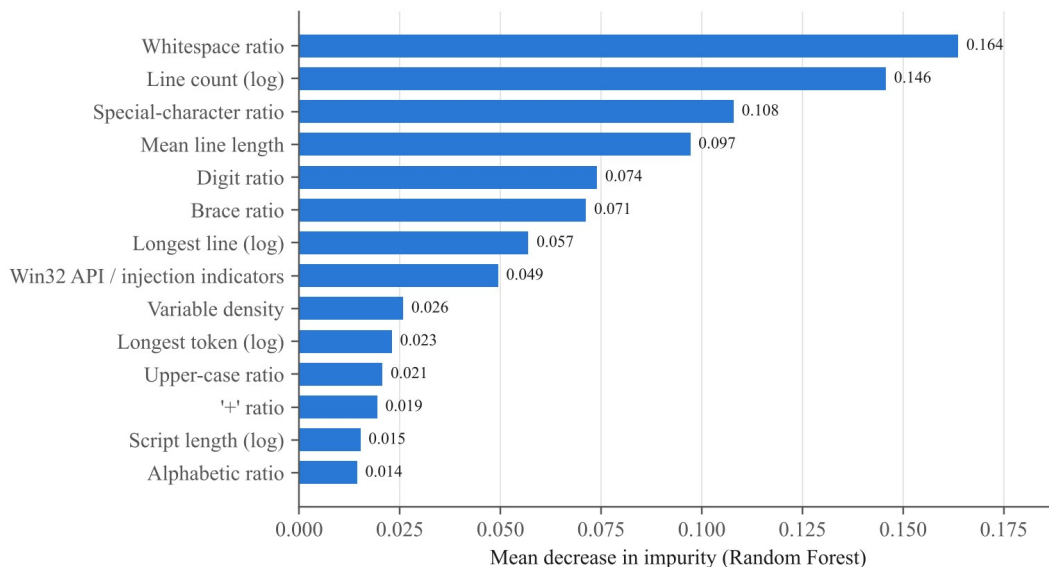


Figure 6: Fourteen most important hand-crafted features of the Random Forest (mean decrease in impurity)

5.6 Error Analysis

Looking at the ensemble’s 30 false positives and 72 false negatives (without reproducing their content) reveals two distinct patterns:

- **False positives are long, dual-use tools.** Their median length is 47,580 characters, more than twenty times that of a typical benign script, and on average 3.13 indicator groups fire per script (versus 0.32 for correctly classified benign scripts). 70% use reflection and 63% call Win32 APIs. Inspection of their function names shows code-signing helpers, memory and logon-session inspection utilities, DLL-loading wrappers and penetration-

testing modules that the dataset labels benign. Such scripts are legitimately ambiguous: the same code that an administrator uses for diagnostics is used by attackers for injection.

- **False negatives look like ordinary administration.** They contain few behavioural indicators (1.36 groups on average; 21% trigger none). Typical examples write registry values, create a scheduled task with an innocuous name, collect the user and host name, or build strings from character codes. Whether such actions are malicious depends on context—who ran them, from where, and what happened next—that is not visible in the script text alone.

The tiered policy softens the impact of both error types. 45.8% of the missed malicious scripts still score at least 0.30 and therefore raise an ALERT for an analyst rather than passing silently, while 60% of the false positives fall below the CONTAIN threshold and only generate a ticket. Adding context from process ancestry or network telemetry, as in provenance-based triage [8], [35], is the most promising way to reduce both error types further.

5.7 Automated Response Simulation

To test the complete detection-to-response loop, the ensemble was trained on four of the five family-aware folds and the fifth fold was replayed as a stream of synthetic Event ID 4104 records. Each record carried a script-block identifier, host name, user, process ID and, for 60% of events, a file path. The replay used the base ensemble trained on standalone scripts, so it measures the response loop on ordinary scripts; the embedded-code case is covered in Section 5.4. Table 11 summarises the outcome.

Table 11: Outcome of replaying an unseen fold as Event ID 4104 records through the response engine (dry-run)

Measure	Value
Training scripts (folds 1–4) / replayed events (fold 5)	4,944 / 1,233
Replayed malicious / benign	373 / 860
Malicious: ALLOW / ALERT / CONTAIN / ISOLATE	11 / 22 / 178 / 162
Benign: ALLOW / ALERT / CONTAIN / ISOLATE	851 / 7 / 2 / 0
Malicious scripts contained (CONTAIN or ISOLATE)	91.15%
Malicious scripts at least alerted	97.05%
Benign scripts wrongly contained	0.23%
Benign scripts sent to analyst as ALERT	0.81%
Isolations awaiting analyst approval	162
Mean actions per contained event	5.41
Latency per event: median / mean / 95th percentile	90.7 / 100.6 / 144.3 ms
Throughput (single process)	9.9 events per second

Figure 8 visualises the outcome. Of the 373 malicious events, 340 (91.2%) were contained automatically—178 at the CONTAIN tier and 162 at the ISOLATE tier awaiting analyst approval—and a further 22 raised an ALERT, so only 11 (2.9%) passed without any signal reaching an analyst. On the benign side, 851 of 860 events (99.0%) were allowed silently, 7 produced an alert and only 2 (0.23%) were wrongly contained. Figure 7 explains why the tiers work: the out-of-fold risk scores of benign scripts are concentrated close to 0 and those of malicious scripts close to 1, and the ALERT band between 0.30 and 0.70 catches most of the

genuinely uncertain cases, so that automated containment is applied almost only where the model is confident. The large number of isolation requests reflects how common process-injection loaders are in this dataset; in practice, approvals would be grouped per host, because several events usually come from the same compromised machine. Note that latency was measured in a shared four-core container while other experiments were running, so it is a conservative figure.

Every contained event carried ATT&CK context. The techniques attached most often were T1059.001 (187 events), T1620 (175 events), T1055 (174 events), T1105 (134 events), which matches the shellcode-loader and download-cradle families that dominate the malicious data. The median end-to-end latency of 90.7 ms per event—dominated by feature extraction and scoring—means that a single process can keep up with roughly 35,675 script blocks per hour; long scripts take longer, as the 95th percentile of 144.3 ms shows, but remain far below the time an attacker needs to move laterally.

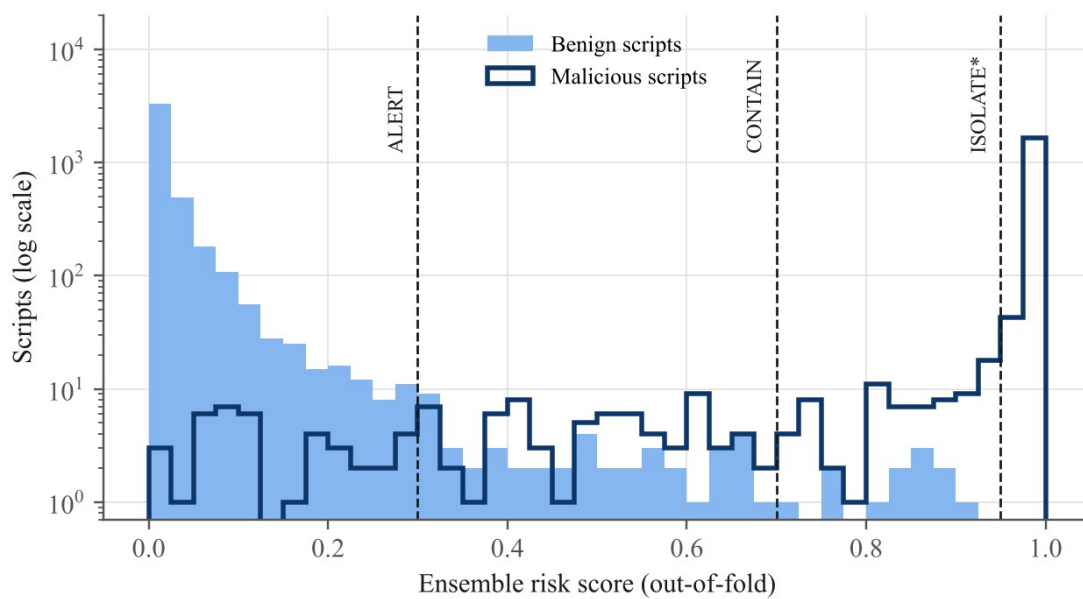


Figure 7: Distribution of out-of-fold ensemble risk scores for benign and malicious scripts, with the policy thresholds



Figure 8: Response tiers assigned to benign and malicious events in the replay experiment

5.8 Comparison with Published Work

Table 12 places the results next to published work. On the same corpus, our accuracy under the stricter family-aware protocol (98.35%) is close to the 98.93% reported by Fang et al. with random splits, even though our models are simpler and do not use AST parsing or embeddings. Under a naive split comparable to theirs our ensemble reaches 98.83%. The comparison with Hendler et al. is only indicative, because their data come from real AMSI telemetry with far more benign variety, which is the harder and more realistic setting.

Table 12: Indicative comparison with published PowerShell detectors (datasets and protocols differ, so the figures are not directly comparable)

Study	Data / protocol	Reported result
Fang et al. 2021 [12]	MPSD; random 5-fold CV	Accuracy 98.93% (original), 97.76% (mixed)
Song et al. 2021 [25]	Own corpus; token/AST features	Detection rate $\approx 98\%$
Hendler et al. 2020 [22]	AMSI data from Microsoft	TPR $\approx 90\%$ at FPR $< 0.1\%$
This work (ensemble)	MPSD, de-duplicated; family-aware 5-fold CV	Accuracy 98.35%, F1 0.9725, TPR 97.39% at 1% FPR
This work (ensemble, naive split)	MPSD raw; random 5-fold CV	Accuracy 98.83%, F1 0.9880

5.9 Limitations and Threats to Validity

- **Dataset scope.** MPSD is a single public corpus with a limited number of malicious families and benign scripts drawn mainly from public repositories. Enterprise environments contain in-house scripts that may look different, so the false-positive rate should be re-measured on local data before deployment.
- **No temporal split.** The scripts carry no reliable timestamps, so it was not possible to train on older and test on newer attacks, as recommended by Pendlebury et al. [32]. Family-aware splitting addresses near-duplicates but not concept drift.
- **Static view only.** The detector sees only script text. Context such as parent process, user, network connections and command-line arguments, which EDR products collect, would help with the ambiguous cases identified in Section 5.6.
- **Adversarial robustness.** An attacker who can query the model could search for variants that lower the score [36]. Using AMSI or Event ID 4104 output (which already removes several obfuscation layers), combining lexical and behavioural evidence, and keeping thresholds conservative reduce but do not remove this risk.
- **Simulated response.** Actions were simulated in dry-run mode on synthetic event metadata; the real cost of terminating processes or isolating hosts in a production network was not measured.

6. Conclusion and Future Work

This case study set out to build and evaluate an AI-based system that detects malicious PowerShell scripts and responds to them automatically, while staying honest about how well such a system really works. A careful audit of the public MPSD corpus came first and proved essential: more than half of its malicious files were duplicates, a handful of files carried contradictory labels, byte-order marks offered a spurious shortcut, and the remaining malicious

scripts formed only 816 structural families. Under a family-aware evaluation that respects these facts, a lightweight ensemble of a character n-gram logistic-regression model and a Random Forest on 33 interpretable features reached an F1-score of 0.9725, a ROC-AUC of 0.9982 and a true-positive rate of 97.39% at a 1% false-positive rate. A naive random split would have understated the miss rate roughly 2-fold, which is a warning for anyone comparing published numbers. The study also exposed a clear weakness: malicious code hidden inside long benign scripts evaded models trained only on standalone samples about half of the time, because whole-script shape features are diluted by the benign host. Adding such examples to the training data raised the ensemble’s detection of embedded payloads to 96.4%, at the cost of a small rise in false positives on ordinary scripts.

Detection was then connected to action. The tiered response engine consumes the same Event ID 4104 records that Windows already produces, attaches ATT&CK techniques and defanged indicators to every decision, and scales its response from logging to process termination, quarantine and network blocking, with host isolation held back for analyst approval. In a replay of unseen scripts it contained 91.2% of malicious events, wrongly contained 0.23% of benign ones and needed a median of 90.7 ms per event. These results show that explainable machine learning combined with a proportionate, auditable response policy is a practical way to defend against living-off-the-land PowerShell attacks, in line with current guidance to keep PowerShell but monitor it closely [5], [11].

Several extensions would strengthen the system:

1. Validate on enterprise telemetry and with a temporal split to measure concept drift, and add periodic retraining driven by analyst feedback.
2. Score individual functions and script blocks separately, in addition to the whole script, so that a small malicious section cannot be diluted by a large benign host.
3. Add AST-based features and pre-trained code embeddings [12], [22] and compare them with the current features under the same family-aware protocol.
4. Integrate the model as an AMSI provider so that high-confidence scripts are blocked before execution rather than after.
5. Enrich decisions with process-tree and network context from EDR provenance graphs to resolve the dual-use cases that cause most errors [35].
6. Use a large language model as a second-stage analyst assistant that explains ALERT-tier scripts in natural language [10], keeping the fast classical model as the first-line filter.
7. Evaluate robustness against adaptive adversaries and harden the model with adversarially augmented training data.

References

- [1] C. Wueest and H. Anand, “Living off the land and fileless attack techniques,” Symantec Internet Security Threat Report (ISTR) Special Report, Symantec Corp., Jul. 2017.
- [2] C. Wueest, “The increased use of PowerShell in attacks,” Symantec Security Response White Paper, Symantec Corp., Oct. 2016.
- [3] The MITRE Corporation, “Command and Scripting Interpreter: PowerShell (T1059.001),” *MITRE ATT&CK Enterprise Matrix*. [Online]. Available: <https://attack.mitre.org/techniques/T1059/001/>
- [4] Red Canary, “2025 Threat Detection Report,” Red Canary Inc., Denver, CO, USA, 2025. [Online]. Available: <https://redcanary.com/threat-detection-report/>

- [5] National Security Agency, Cybersecurity and Infrastructure Security Agency, New Zealand National Cyber Security Centre, and UK National Cyber Security Centre, “Keeping PowerShell: Security measures to use and embrace,” Joint Cybersecurity Information Sheet, Jun. 2022.
- [6] D. Bohannon, “Invoke-Obfuscation: PowerShell command and script obfuscator,” presented at DerbyCon 6.0, Louisville, KY, USA, 2016. [Online]. Available: <https://github.com/danielbohannon/Invoke-Obfuscation>
- [7] D. Bohannon and L. Holmes, “Revoke-Obfuscation: PowerShell obfuscation detection using science,” White Paper, Black Hat USA, Las Vegas, NV, USA, Jul. 2017.
- [8] W. U. Hassan, S. Guo, D. Li, Z. Chen, K. Jee, Z. Li, and A. Bates, “NoDoze: Combatting threat alert fatigue with automated provenance triage,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2019.
- [9] C. Patsakis, F. Casino, and N. Lykousas, “Assessing LLMs in malicious code deobfuscation of real-world malware campaigns,” *Expert Systems with Applications*, vol. 256, Art. no. 124912, 2024.
- [10] M. Wang, E. Topolovec, B. Arana, N. V. P. S. G. C. Kuladeep, Z. Xi, G. Yan, X. Shu, S. D. Stoller, and P. Yang, “Detection and explanation of PowerShell malware with large language models,” Manuscript, Binghamton University, IBM T. J. Watson Research Center and Stony Brook University, 2026. [Online]. Available: <https://www3.cs.stonybrook.edu/~stoller/papers/powershell-malware-2026.pdf>
- [11] A. Nelson, S. Rekhi, M. Souppaya, and K. Scarfone, “Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile,” NIST Special Publication 800-61 Rev. 3, National Institute of Standards and Technology, Gaithersburg, MD, USA, Apr. 2025.
- [12] Y. Fang, X. Zhou, and C. Huang, “Effective method for detecting malicious PowerShell scripts based on hybrid features,” *Neurocomputing*, vol. 448, pp. 30–39, 2021, doi: 10.1016/j.neucom.2021.03.117.
- [13] Microsoft, “about_Locking_Windows,” PowerShell Documentation, Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_locking_windows
- [14] Microsoft, “Antimalware Scan Interface (AMSI),” Win32 Apps Documentation, Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/amsi/antimalware-scan-interface-portal>
- [15] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, “MITRE ATT&CK: Design and philosophy,” The MITRE Corporation, McLean, VA, USA, Tech. Rep. MTR180360, 2018.
- [16] D. Ugarte, D. Maiorca, F. Cara, and G. Giacinto, “PowerDrive: Accurate de-obfuscation and analysis of PowerShell malware,” in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2019)*, Lecture Notes in Computer Science, vol. 11543. Cham, Switzerland: Springer, 2019, pp. 240–259.
- [17] Z. Li, Q. A. Chen, C. Xiong, Y. Chen, T. Zhu, and H. Yang, “Effective and light-weight deobfuscation and semantic-aware attack detection for PowerShell scripts,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, London, UK, 2019, pp. 1831–1847.
- [18] C. Xiong, Z. Li, Y. Chen, T. Zhu, J. Wang, H. Yang, and W. Ruan, “Generic, efficient, and effective deobfuscation and semantic-aware attack detection for PowerShell scripts,” *Frontiers of Information Technology & Electronic Engineering*, vol. 23, no. 3, pp. 361–381, 2022, doi: 10.1631/FITEE.2000436.

- [19] M.-H. Tsai, C.-C. Lin, Z.-G. He, W.-C. Yang, and C.-L. Lei, "PowerDP: De-obfuscating and profiling malicious PowerShell commands with multi-label classifiers," *IEEE Access*, vol. 11, 2023.
- [20] R. Li, C. Zhang, H. Chai, L. Ying, H. Duan, and J. Tao, "PowerPeeler: A precise and general dynamic deobfuscation method for PowerShell scripts," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS '24)*, Salt Lake City, UT, USA, 2024.
- [21] D. Hendler, S. Kels, and A. Rubin, "Detecting malicious PowerShell commands using deep neural networks," in *Proc. ACM Asia Conf. Computer and Communications Security (ASIACCS)*, Incheon, Republic of Korea, 2018, pp. 187–197.
- [22] D. Hendler, S. Kels, and A. Rubin, "AMSI-based detection of malicious PowerShell code using contextual embeddings," in *Proc. 15th ACM Asia Conf. Computer and Communications Security (ASIA CCS '20)*, Taipei, Taiwan, 2020, pp. 679–693.
- [23] Microsoft Defender Research Team, "Deep learning rises: New methods for detecting malicious PowerShell," Microsoft Security Blog, Sep. 3, 2019. [Online]. Available: <https://www.microsoft.com/en-us/security/blog/2019/09/03/deep-learning-rises-new-methods-for-detecting-malicious-powershell/>
- [24] G. Rusak, A. Al-Dujaili, and U.-M. O'Reilly, "AST-based deep learning for detecting malicious PowerShell," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Toronto, ON, Canada, 2018, pp. 2276–2278.
- [25] J. Song, J. Kim, S. Choi, J. Kim, and I. Kim, "Evaluations of AI-based malicious PowerShell detection with feature optimizations," *ETRI Journal*, vol. 43, no. 3, pp. 549–560, 2021.
- [26] M. Mimura and Y. Tajiri, "Static detection of malicious PowerShell based on word embeddings," *Internet of Things*, vol. 15, Art. no. 100404, 2021.
- [27] J. Saxe and K. Berlin, "eXpose: A character-level convolutional neural network with embeddings for detecting malicious URLs, file paths and registry keys," arXiv preprint arXiv:1702.08568, 2017.
- [28] D. Ucci, L. Aniello, and R. Baldoni, "Survey of machine learning techniques for malware analysis," *Computers & Security*, vol. 81, pp. 123–147, 2019.
- [29] D. Gibert, C. Mateu, and J. Planes, "The rise of machine learning for detection and classification of malware: Research developments, trends and challenges," *Journal of Network and Computer Applications*, vol. 153, Art. no. 102526, 2020.
- [30] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, USA, 2010, pp. 305–316.
- [31] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, 2022, pp. 3971–3988.
- [32] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. 28th USENIX Security Symp.*, Santa Clara, CA, USA, 2019, pp. 729–746.
- [33] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, Art. no. e0118432, 2015.
- [34] C. Islam, M. A. Babar, and S. Nepal, "A multi-vocal review of security orchestration," *ACM Computing Surveys*, vol. 52, no. 2, Art. no. 37, 2019.

- [35] W. U. Hassan, A. Bates, and D. Marino, “Tactical provenance analysis for endpoint detection and response systems,” in *Proc. IEEE Symp. Security and Privacy (S&P)*, San Francisco, CA, USA, 2020, pp. 1172–1189.
- [36] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” in *Proc. Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.
- [37] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [38] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [39] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [40] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [41] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 3146–3154.
- [42] R. Kohavi, “A study of cross-validation and bootstrap for accuracy estimation and model selection,” in *Proc. 14th Int. Joint Conf. Artificial Intelligence (IJCAI)*, Montreal, QC, Canada, 1995, pp. 1137–1143.
- [43] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 4765–4774.